

The Kernel Guide

How the Gated Demand Architecture kernel works, how to use it,

and why it differs from other frameworks

· Guide bilingue — English & Français ·

PHP 8.2+

Gated Demand Architecture

Contributors

App Builders

Step-by-step

EN / FR

Package: **alfacode-team/php-service-platform** · Namespace:

AlfacodeTeam\PhpServicePlatform\Kernel\

Audience: Internal contributors & framework users · Expanded edition, version 2.0

Table of Contents

- 1. What the Kernel Is & Why It Exists**
- 2. Why GDA — Benefits & Differences vs Other Frameworks**
- 3. The Kernel Lifecycle — configure → build → materialize**
- 4. App Builders: Using the Kernel Step by Step**
 - 4.1 The fluent builder & what each method does
 - 4.2 Ports vs essential vs on-demand modules
 - 4.3 Entry points
 - 4.4 TUTORIAL: build a module from scratch (8 steps)
 - 4.5 Worker jobs & CLI commands
 - 4.6 Testing a service
- 5. Contributors: How the Kernel Works Internally**
 - 5.1 build() vs materialize()
 - 5.2 The 9-stage boot pipeline
 - 5.3 Trace an HTTP request end to end
 - 5.4 Containers & scope enforcement
 - 5.5 WALKTHROUGH: add a new port / add a boot stage
 - 5.6 Security gateway internals
- 6. The Five Access Rules & Exception Hierarchy**
- 7. Quick Reference & Checklists**

1. What the Kernel Is & Why It Exists

The **Kernel** is the heart of the AlfacodeTeam PhpServicePlatform framework — a small, domain-agnostic core that boots an application, runs security *before* any business code, loads only the modules a given request needs, and routes work through three pipelines: HTTP, CLI, and Worker.

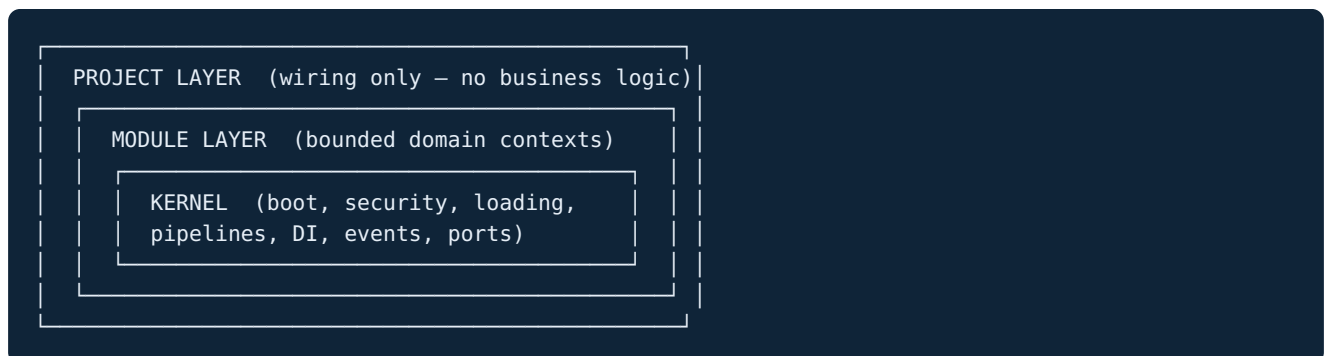
It is built on the **Gated Demand Architecture (GDA)**: security *gates* every request, and modules are loaded on *demand*, so a request pays only for what it actually uses.

The problem it solves

Most full-stack frameworks boot the *entire* application on every request: every service provider registers, every route is compiled, every middleware is constructed — then a request is matched and most of that work is thrown away. GDA inverts this: it compiles a static picture of the app once (manifests), then at request time it loads *only* the modules in that route's dependency graph. A blocked request (failed firewall, bad CSRF, rate limit) costs **zero module work** because security runs before anything loads.

Principle	Meaning
Security before everything	The SecurityGateway runs before any module loads. Denied = zero module cost.
Load only what is needed	Only the modules required for this specific request are loaded.
One module, one domain	Every module owns exactly one business domain.
Isolation by default	Modules cannot reach each other's internals — enforced at runtime by scoped containers.
Infrastructure independence	The kernel defines port interfaces; the project provides implementations.
Explicit over implicit	Everything is declared in <code>module.json</code> . Nothing is auto-discovered at runtime.

The three worlds



- **Kernel** — knows nothing about modules or business domains. Changes rarely.
- **Module** — knows nothing about the project. Wires to the kernel through contracts only.
- **Project** — knows everything, contains no business logic. Wiring only.

How to read this guide. App builders / framework users → chapters 3, 4, 6, 7. Internal contributors → everything, especially chapter 5. Both audiences benefit from chapter 2.

2. Why GDA — Benefits & Differences vs Other Frameworks

This chapter is the "why bother" — what GDA buys you that Laravel, Symfony, or Slim do not, and the trade-offs.

2.1 Side-by-side comparison

Concern	Laravel / Symfony (typical)	PhpServicePlatform (GDA)
Boot cost per request	All providers/bundles register every request	Only the route's module graph loads
Cost of a blocked request	App often boots before auth/throttle middleware	Security runs first — denied = 0 module cost
Routes	PHP route files / annotations	Declared in <code>module.json</code> , compiled to a manifest
Module isolation	Convention; any service is globally resolvable	Runtime-enforced — cross-module internals throw
Money / domain modelling	Floats & Eloquent models common	Integer-cents value objects; pure Domain layer
Infrastructure coupling	Facades / global helpers reach the container	Ports + DI only; no global singletons
Swoole / long-running safety	Needs care — global state leaks between requests	Core frozen; request state in a discarded scope
Native distribution	Requires PHP + Composer + vendor/	Ships OS-native packages (.deb / .zip / .tar.gz)

2.2 The concrete benefits

1 — Pay-per-use performance

A request to `/api/tasks` loads the Task module and its `requires[]` (e.g. the Database module) — and nothing else. The Invoice, Payment, or Auth modules are never instantiated unless the route graph pulls them in. Under OpenSwoole the `CoreContainer` (ports + kernel services) is built once per worker; only the cheap request-scoped `ModuleContainer` is rebuilt per request.

2 — Security that cannot be bypassed

Because the SecurityGateway is a pipeline stage that runs *before* ResolveStage/LoadStage, a 403/429 short-circuits the request with no module ever constructed. You cannot accidentally "forget the auth middleware on a route" — there is no per-route opt-in to forget.

3 — Architecture enforced by the runtime, not by code review

The Five Access Rules are not a style guide — `ModuleContainer::bindInternal()` physically throws `ScopeViolationException` if a controller reaches into another module's repository. Bad layering fails fast, in tests, instead of rotting over years.

4 — Testability by construction

Services depend on ports and contracts, never on concrete infrastructure, so a unit test swaps in `InMemoryRepository` / `FakeTransactionManager` with no framework bootstrapping at all.

5 — Honest deployment

The kernel can be shipped as an OS-native package with no `vendor/` directory, because env loading, the error net, and the CLI runtime are all dependency-light first-party code.

The trade-off (be honest): GDA asks for more up-front structure — one domain per module, contracts at boundaries, manifests, declared config. For a 3-page CRUD site this is overhead. The payoff appears when the system grows past the point where a "just put it in a controller" framework becomes a tangle. Choose GDA for platforms, not for throwaway prototypes.

3. The Kernel Lifecycle

Three distinct phases explain nearly every performance and correctness property of the framework.

Phase	Method	What happens
1. Configure	<code>Kernel::configure()</code> + <code>with*()</code>	Collect ports, security layers, modules, error pipeline, paths. Pure declaration — nothing is constructed.
2. Build	<code>->build()</code>	Runs the BootPipeline: validates config and compiles manifests (service / route / job / command). Compile-only — no pipeline built, no module wired, core not frozen.
3. Materialize	first <code>http()</code> / <code>cli()</code> / <code>workerLoop()</code>	Constructs the pipelines, wires every module ONCE (<code>Provider::boot</code>), exposes kernel services, then freezes the core. Runs at most once.

Why split build and materialize? `build()` is cheap and safe enough to run in CI or to warm caches. `materialize()` is lazy and surface-specific: an HTTP-only process never reads the job manifest; a CLI process never reads the route manifest. The first entry-point call decides the `RuntimeMode`.

```
Kernel::configure()    // phase 1 – declare
  ->withPorts([...]) ->withModules([...])
  ->build();            // phase 2 – validate + compile manifests (no pipelines yet)

$kernel->http();        // phase 3 – materialize(RuntimeMode::Http), return HttpPipeline
$kernel->mode();        // RuntimeMode::Http (null before any entry point)
```

4. App Builders: Using the Kernel Step by Step

Audience: app builders / framework users.

4.1 The fluent builder & what each method does

A shared `base.php` returns an *unbuilt* builder; each project extends it and calls `->build()`.

app/bootstrap/base.php – shared base (NO `->build()`)

```
return Kernel::configure()
    ->withBasePath(dirname(__DIR__, 2))          // var/, userdata/ live under here
    ->withPorts([
        DatabasePort::class => fn() => new PDODatabase(dsn: env('DB_DSN')),
        CachePort::class    => fn() => new InMemoryCache(),
    ])
    ->withSecurity([ new CsrfTokenLayer(exemptPaths: ['/api']) ])
    ->withModules([ CryptoProvider::class, DatabaseProvider::class ])
    ->withEssentialModules([ SessionProvider::class, CookieProvider::class ]);
```

projects/admin/bootstrap/app.php – concrete project

```
$builder = require __DIR__ . '/../../app/bootstrap/base.php';
return $builder
    ->withProjectPath(dirname(__DIR__))           // isolates this project's var/ + userdata/
    ->withModules([ AuthModule::class, InvoiceModule::class ])
    ->build();                                     // validate + compile manifests
```

Method	Merge semantics	Use for
<code>withPorts([...])</code>	Merges; later key wins	Infra implementations. A Closure is a lazy singleton — built on first use only.
<code>withSecurity([...])</code>	Appends	Security layers, cheapest first (firewall → rate-limit → CSRF → auth).
<code>withModules([...])</code>	Appends + de-dupes, keeps order	On-demand modules; loaded only when a route's dependency graph pulls them in.
<code>withEssentialModules([...])</code>	Appends + de-dupes	Request-scoped infra needed app-wide (sessions, cookies). Adds a <code>register()</code> cost per request.

4.2 Ports vs essential vs on-demand modules — the decision

Rule of thumb: stateless + always → a **port**. Stateful + always → an **essential module**. Needed by some routes → an **on-demand module** with `requires[]` in its `module.json`.

Choice	Lifetime	Example
--------	----------	---------

Port (CoreContainer)	App-lifetime, one per worker	Database, hashing, encryption, in-memory cache
Essential module	register() every request	Session, cookies, Redis cache override
On-demand module	register() only when in the route graph	Invoice, Task, outbound HTTP client

Never put request-scoped state on a port (CoreContainer) — it would leak across requests under OpenSwoole.

4.3 Entry points

```
$kernel->http()           // → HttpPipeline::handle(Request): Response
$kernel->cli()             // → CliPipeline::run(argv): int
$kernel->workerLoop()     // → WorkerLoop::run(callable $puller, int $maxIterations = 0)
$kernel->container()      // → CoreContainer (materializes as the Cli surface)
$kernel->mode()           // → ?RuntimeMode (null until first entry point)
$kernel->requestTeardown() // call after each request under OpenSwoole (no-op on FPM/CLI)
```

app/public_html/index.php – FPM entry

```
$kernel = require EntryHelpers::bootstrapPathFor($rootPath, $project);
$request = Request::capture();
$kernel->http()->handle($request)->send();
```

4.4 TUTORIAL — Build a module from scratch (8 steps)

We rebuild the real **Task** plugin. It lives in `plugins/Task/` under the `Plugins\Task\` namespace and owns one domain: `task.management`.

Step 1 — Declare the module in module.json (single source of truth)

plugins/Task/module.json

```
{
  "name": "task", "version": "1.0.0",
  "solves": "task.management", "type": "module",
  "requires": ["database.query"],
  "exposes": ["Plugins\\Task\\API\\Contracts\\TaskServiceContract"],
  "routes": [
    { "method": "GET", "path": "/api/tasks", "handler": "Plugins\\Task\\Infrastructure\\Http\\TaskController@index" },
    { "method": "POST", "path": "/api/tasks", "handler": "Plugins\\Task\\Infrastructure\\Http\\TaskController@create" },
    { "method": "GET", "path": "/api/tasks/{id}", "handler": "Plugins\\Task\\Infrastructure\\Http\\TaskController@show" },
    { "method": "POST", "path": "/api/tasks/{id}/complete", "handler": "...TaskController@complete" }
  ],
  "emits": ["task.created"], "config": []
}
```

Routes are declared HERE ONLY. Every env var the module reads must appear in `config[]` or boot fails. Plugin handlers use the full `Plugins\...` class path.

Step 2 — The Domain entity (zero external imports)

plugins/Task/Domain/Entities/Task.php

```
final class Task
{
    private array $domainEvents = [];

    private function __construct(           // ALWAYS private
        private readonly TaskId $id,
        private string $title,
        private TaskStatus $status,
    ) {}

    public static function create(string $title): self // named ctor – records events
    {
        $t = new self(TaskId::generate(), $title, TaskStatus::PENDING);
        $t->domainEvents[] = new TaskCreatedDomainEvent($t->id, $title, new
\DateTimeImmutable());
        return $t;
    }

    public function markCompleted(): void {           // state transition
        if ($this->status === TaskStatus::COMPLETED) {
            throw new \DomainException('Task already completed');
        }
        $this->status = TaskStatus::COMPLETED;
    }

    public function releaseEvents(): array {           // returns AND clears
        $e = $this->domainEvents; $this->domainEvents = []; return $e;
    }
}
```

Step 3 — Published contract (the only thing other modules may import)

plugins/Task/API/Contracts/TaskServiceContract.php

```
interface TaskServiceContract
{
    public function list(): array;
    public function create(CreateTaskDTO $dto): TaskDTO;
    public function find(string $id): ?TaskDTO;
    public function complete(string $id): ?TaskDTO;
    public function delete(string $id): bool;
}
```

Step 4 — Repository (DatabasePort only, tenant-scoped)

```
final class TaskRepository
{
    public function __construct(
        private readonly DatabasePort $db,           // ONLY external dependency
        private readonly Identity $identity, // for tenant scoping
    ) {}
}
```

```

public function find(string $id): ?Task {
    try {
        $row = $this->db->queryOne(
            'SELECT * FROM tasks WHERE id = :id AND tenant_id = :t',
            ['id' => $id, 't' => $this->identity->tenantId]);
    } catch (\PDOException $e) {
        throw new RepositoryException("find task [{$id}]", layer: 'repository.task',
previous: $e);
    }
    return $row === null ? null : TaskHydrator::hydrate($row);
}
}

```

Step 5 — Application service (the mandatory transaction + event shape)

This is the real `TaskService::create()`. Note the exact ordering: begin collection → begin tx → work → commit; on failure rollback *and* discard; dispatch integration events **ONLY** after commit, never inside `try{}`.

plugins/Task/Application/Services/TaskService.php

```

public function create(CreateTaskDTO $dto): TaskDTO
{
    $this->collector->beginCollection();
    $this->transaction->begin();
    try {
        $task = Task::create($dto->title);
        foreach ($task->releaseEvents() as $event) {
            $this->collector->collect($event);
        }
        $this->repository->save($task);
        $this->transaction->commit();
    } catch (\Throwable $e) {
        $this->transaction->rollback();
        $this->collector->discard(); // ← no phantom events on rollback
        throw new ServiceException('task.create.failed', layer: 'service.task', previous: $e);
    }

    // AFTER commit only – never inside the try block
    $created = $this->firstCreatedEvent($this->collector->release());
    if ($created !== null) {
        $this->eventBus->dispatch(new TaskCreatedIntegrationEvent(
            taskId: $created->taskId->value(),
            title: $created->title,
            occurredAt: $created->occurredAt->format(\DateTimeInterface::RFC3339),
        ));
    }
    return TaskDTO::fromEntity($task);
}

```

Step 6 — Controller (max 3 lines per method)

plugins/Task/Infrastructure/Http/TaskController.php

```

final class TaskController
{
    public function __construct(private readonly TaskServiceContract $tasks) {} // contract only

    public function create(Request $request): Response {

```

```

        $task = $this->tasks->create(CreateTaskDTO::fromRequest($request));
        return Response::json(['data' => $task->toArray()], 201);
    }

    public function show(Request $request, string $id): Response {
        $task = $this->tasks->find($id);
        return $task === null
            ? Response::notFound("Task [{id}] not found.")
            : Response::json(['data' => $task->toArray()]);
    }
}

```

Step 7 — Provider: register() bindings + boot() hooks

plugins/Task/Provider.php

```

final class Provider implements ModuleContract
{
    public function solves(): string { return 'task.management'; }
    public function requires(): array { return [DatabasePort::class]; }
    public function exposes(): array { return [TaskServiceContract::class]; }

    public function register(ModuleContainer $c): void {
        // internal – ScopeViolationException if resolved from outside this module
        $c->bindInternal(TaskRepository::class, static fn(ModuleContainer $c) =>
            new TaskRepository($c->make(DatabasePort::class), $c->make(Identity::class)));
        // public – resolvable by modules that 'require' task.management
        $c->bind(TaskServiceContract::class, static fn(ModuleContainer $c) =>
            new TaskService(
                repository: $c->make(TaskRepository::class),
                transaction: $c->make(TransactionManager::class),
                collector: $c->make(DomainEventCollector::class),
                eventBus: $c->make(EventBus::class),
                identity: $c->make(Identity::class),
            ));
    }

    public function boot(HttpPipeline $http, CliPipeline $cli,
        WorkerPipeline $worker, EventBus $events): void {
        // hooks + event subscriptions go here, e.g.:
        // $events->subscribe('payment.succeeded', MarkTaskPaidListener::class);
    }
}

```

Step 8 — Register the provider with the project

```

// projects/admin/bootstrap/app.php
return $builder
    ->withModules([ \Plugins\Task\Provider::class ])
    ->build();

```

That is the whole module. `build()` validates it (route handler format, no duplicate `solves`, declared config) and compiles it into the manifests. The first HTTP request to `/api/tasks` loads it on demand.

4.5 Worker jobs & CLI commands

A job

```
// module.json
{ "type": "job", "queue": "emails",
  "retry": { "max": 3, "strategy": "exponential", "jitter": true },
  "timeout": 30, "requires": ["mail.port", "invoice.generation"] }

final class SendInvoiceEmailJob implements JobContract
{
    public function handle(JobPayload $payload): JobResult {
        // ... throw to trigger a retry
        return JobResult::success(['sent' => 1]);
    }
    public function failed(JobPayload $payload, \Throwable $e): void { /* after max retries */ }
}
```

A CLI command

```
final class SyncTasksCommand extends AbstractCommand // php-io-cli, NOT Symfony
{
    protected string $signature = 'tasks:sync';
    public function handle(): int {
        $this->output->success('Synced.');
```

```
        return 0;
    }
}
// register: $cli->command(SyncTasksCommand::class); // resolved via CoreContainer → ports injectable
```

4.6 Testing a service (no real infrastructure)

```
$repo      = new InMemoryTaskRepository();
$txn       = new FakeTransactionManager();
$bus       = new FakeIntegrationEventBus();
$collector = new DomainEventCollector();
$identity  = Identity::asUser('user-1', 'tenant-abc');

$sut = new TaskService($repo, $txn, $collector, $bus, $identity);
$result = $sut->create(new CreateTaskDTO('Write the guide'));

$this->assertTrue($txn->wasCommitted());
$bus->assertDispatched(TaskCreatedIntegrationEvent::class, times: 1);

// rollback path:
$repo->failOnNextSave();
try { $sut->create($dto); } catch (ServiceException) {}
$this->assertTrue($txn->wasRolledBack());
$bus->assertNotDispatched(TaskCreatedIntegrationEvent::class); // no phantom event
```

5. Contributors: How the Kernel Works Internally

Audience: internal contributors. This chapter explains the machinery inside `src/Kernel/`.

5.1 build() vs materialize()

`Kernel::build()` creates a `CoreContainer`; registers each port as a lazy `singleton` (Closure) or eager `instance` (object); then runs the `BootPipeline` to validate config + compile manifests. The kernel is marked `built` but **not** frozen, and **no** pipeline exists yet.

`Kernel::materialize(RuntimeMode)` (private, once): builds `EventBus`, `WorkerPipeline`, `HttpPipeline` (with a `SecurityGateway` over the layers), `CliPipeline`, `WorkerLoop`; exposes them back into the core; loops over every module, instantiates the provider, calls `boot()` to register hooks + subscriptions on the live instances; then calls `core->freeze()`.

All three pipelines are constructed at materialize because a module's single `boot()` may hook any of them — but each pipeline is a cheap shell that defers its manifest disk I/O until its own first run. So an HTTP process never reads the job/command manifest in practice.

5.2 The 9-stage boot pipeline (fail-fast)

#	Stage	Checks / produces
1	ValidateConfig	All declared env vars present + typed
2	DetectConflicts	No two modules <code>solves()</code> the same domain
3	DetectCycles	No circular <code>requires[]</code>
4	CompileServiceManifest	<code>service-manifest.php</code>
5	CompileRouteManifest	<code>route-manifest.php</code> (validates <code>Controller@method</code>)
6	CompileJobManifest	<code>job-manifest.php</code>
7	CompileCommandManifest	<code>command-manifest.php</code>
8	RegisterPorts	Verify port bindings exist
9	BindSecurity	Verify security layers

5.3 Trace an HTTP request end to end

Follow `POST /api/tasks` through the running kernel:

1. **CorrelationIdStage** — generates/propagates `X-Correlation-ID` so the whole request is traceable in logs.

2. **SecurityStage** — runs the `SecurityGateway` over its layers (firewall → rate-limit → CSRF → auth). On `deny` it returns an HTTP error immediately — **no module is loaded**. On `allow` it attaches the `Identity` to the request. `after.security` hooks run here.
3. **ResolveStage** — looks up `POST /api/tasks` in `route-manifest.php` via `RouteMatcher`, yielding the service/domain (`task.management`) and handler `TaskController@create`.
4. **LoadStage** — `DependencyGraphCalculator` walks the service manifest to compute the transitive `requires[]` (Task → Database). `OnDemandLoader` builds a fresh `ModuleContainer`, registers the request-scoped kernel services (`Identity`, `DomainEventCollector`, `TransactionManager`) under public scope, then runs each module's `register()` under its own scope so `bindInternal()` isolation holds. `after.load` hooks run here.
5. **ExecuteStage** — resolves `TaskController` via `makeInScope(..., 'task.management')`, builds the DTO from the request, calls the controller method, gets a `Response`. `after.execute` hooks run here.
6. **ErrorStage** — wraps the whole chain; any `Throwable` is classified and routed through the `ErrorPipeline` to notifiers, returning a safe HTTP response.
7. The `ModuleContainer` is discarded — **zero state leaks**. Under Swoole, `requestTeardown()` is called.

Hook priority bands: 1–9 system, 10–19 security, 40–59 feature, 80–99 observability.

5.4 Containers & scope enforcement

Both extend `PHPShots\Common\Container` (the bind-it engine): reflection autowiring, contextual bindings, PSR-11.

- **CoreContainer** — app-lifetime, one per worker. Holds ports + kernel services. `freeze()` locks it after materialize; any write then throws `LogicException`. `getInstance()/setInstance()` are disabled — no global singleton.
- **ModuleContainer** — request-scoped, new per request/job, discarded after. `bindInternal()` enforces scope: resolving an internal binding from outside its owning module throws `ScopeViolationException`. `makeInScope($class, $domain)` resolves with an explicit caller scope (used by ExecuteStage). Call `reset()` at end-of-request under Swoole.

```
// From OnDemandLoader::loadWithIdentity() – request-scoped kernel services
$container = new ModuleContainer($this->core);
$container->setScope(''); // public scope
$container->singleton(Identity::class, fn() => $identity ?? Identity::guest());
$container->singleton(DomainEventCollector::class, fn() => new DomainEventCollector());
$container->singleton(TransactionManager::class,
    fn() => new TransactionManager($this->core->make(DatabasePort::class)));
// then each module's register() runs under its own scope
```

5.5 WALKTHROUGH — extending the kernel

Adding a new port (e.g. a SearchPort)

1. Define the interface in `src/Kernel/Ports/SearchPort.php` — methods only, no implementation.

2. Provide an adapter in a plugin (e.g. `plugins/Search/Infrastructure/ElasticAdapter.php`) implementing it.
3. Bind it in a project's `base.php`: `->withPorts([SearchPort::class => fn() => new ElasticAdapter(env('ES_URL'))])`.
4. Consuming modules inject `SearchPort` in their repository/gateway via DI. The kernel never references the adapter.

A port is for STATELESS app-lifetime infrastructure only. Anything request-scoped must be an essential or on-demand module, never a port.

Adding a boot validation stage

1. Create a stage under `src/Kernel/Boot/Stages/` implementing the boot-stage contract; throw `BootException` on failure (fail fast).
2. Register it in the `BootPipeline` stage list, in the correct order (validation before compilation).
3. Keep it domain-agnostic — it may only reason about manifests, ports, and config, never a specific business module.

5.6 Security gateway internals

```
Request → FirewallLayer → RateLimiterLayer → CsrfTokenLayer → [Auth layer] → pipeline
           ↓ deny(403)           ↓ deny(429)           ↓ deny(403)           ↓ deny(401)
           ZERO module cost at every denial
```

A layer implements `check(Request): SecurityVerdict` and **never throws** — it returns `allow()` or `deny(code, reason)`. Layers are ordered cheapest-first so the cheapest rejection happens earliest. Authorization (role/permission checks) belongs in the *Service* layer, not the gateway. The kernel ships no JWT validator — token auth is provided by a project's Auth module as a layer hook registered in `boot()`.

6. The Five Access Rules & Exception Hierarchy

Absolute, enforced at runtime by `ModuleContainer::bindInternal()`. Violations throw `ScopeViolationException`.

```
Controller → Service      (published contract interface ONLY)
Service    → Repository AND Gateway (the ONLY layer calling both)
Repository → DatabasePort ONLY   (no HTTP, no vendor SDK)
Gateway    → Vendor SDK ONLY     (no DB, no services)
Domain     → NOTHING EXTERNAL    (zero imports outside Domain/)
```

Layer	Throws	Translates from	HTTP
Domain	<code>\DomainException</code> (built-in)	—	422
Service	<code>ServiceException</code>	any caught Throwable	422/500
Repository	<code>RepositoryException</code>	<code>\PDOException</code>	500
Gateway	<code>GatewayException</code>	ALL vendor exceptions	502
Security	never throws — <code>SecurityVerdict::deny()</code>	—	401/403

Error response body shape is always: `{ "error": { "code", "message", "requestId", "fields"? } }`.

7. Quick Reference & Checklists

App-builder checklist (shipping a feature)

- Routes go in `module.json`, never in PHP.
- Every env var a module reads is declared in its `config[]`.
- Read config with `env()`, never `getenv()`.
- Money is a `Money` value object with integer cents — never float.
- Integration events dispatch only AFTER commit, never inside `try{}`.
- Always `collector->discard()` in the catch block.
- Controllers are max 3 lines: DTO → service → Response.
- Local modules live in `plugins/` with the `Plugins\` namespace, full handler paths.

Contributor checklist (changing the kernel)

- Keep the kernel domain-agnostic — no business logic, no module names.
- New boot validation → add a fail-fast stage to the BootPipeline.
- Respect `freeze()` — no writes to the core after materialize.
- Request-scoped state never lives on CoreContainer (leaks across Swoole requests).
- No static properties in request-scoped classes.
- Translate vendor exceptions at their layer — none escape.

Version Française

Guide du Kernel — Architecture à Demande Contrôlée
(GDA)

Table des matières

1. Ce qu'est le Kernel et pourquoi il existe

2. Pourquoi GDA — bénéfices et différences vs autres frameworks

3. Le cycle de vie — configure → build → materialize

4. Développeurs : utiliser le Kernel étape par étape

4.1 Le constructeur fluide

4.2 Ports vs essentiels vs à la demande

4.3 Points d'entrée

4.4 TUTORIEL : créer un module de zéro (8 étapes)

4.5 Jobs Worker et commandes CLI

4.6 Tester un service

5. Contributeurs : fonctionnement interne du Kernel

5.1 build() vs materialize()

5.2 Le pipeline de démarrage à 9 étapes

5.3 Tracer une requête HTTP de bout en bout

5.4 Conteneurs et portées

5.5 GUIDE PRATIQUE : ajouter un port / une étape

5.6 Intérieur de la passerelle de sécurité

6. Les cinq règles d'accès et la hiérarchie d'exceptions

7. Référence rapide et listes de contrôle

1. Ce qu'est le Kernel et pourquoi il existe

Le **Kernel** est le cœur du framework AlfacodeTeam PhpServicePlatform — un noyau compact et indépendant de tout domaine métier qui démarre une application, exécute la sécurité *avant* tout code métier, charge uniquement les modules nécessaires à une requête, et achemine le travail à travers trois pipelines : HTTP, CLI et Worker.

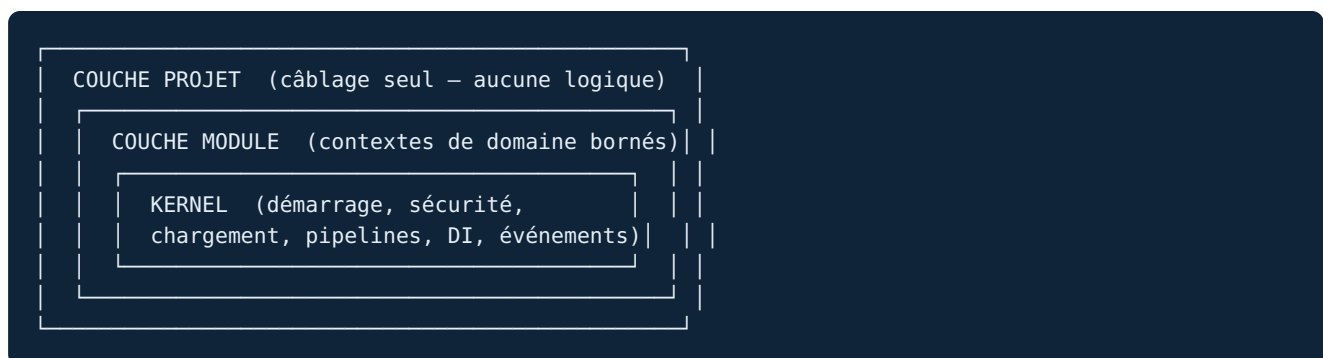
Il repose sur l'**Architecture à Demande Contrôlée (GDA)** : la sécurité *contrôle* chaque requête, et les modules sont chargés à la *demande*, de sorte qu'une requête ne paie que ce qu'elle utilise réellement.

Le problème résolu

La plupart des frameworks complets démarrent l'application *entière* à chaque requête : tous les service providers s'enregistrent, toutes les routes sont compilées, tous les middlewares sont construits — puis une requête est appariée et l'essentiel de ce travail est jeté. GDA inverse cela : il compile une image statique de l'app une fois (manifestes), puis à l'exécution ne charge *que* les modules du graphe de dépendances de la route. Une requête bloquée (pare-feu, CSRF, limite) coûte **zéro travail module** car la sécurité s'exécute avant tout chargement.

Principe	Signification
La sécurité avant tout	La SecurityGateway s'exécute avant tout module. Refusé = coût module nul.
Charger seulement le nécessaire	Seuls les modules requis pour cette requête précise sont chargés.
Un module, un domaine	Chaque module possède exactement un domaine métier.
Isolation par défaut	Les modules n'accèdent pas aux internes des autres — imposé à l'exécution.
Indépendance de l'infrastructure	Le kernel définit des interfaces de ports ; le projet fournit les implémentations.
L'explicite plutôt que l'implicite	Tout est déclaré dans <code>module.json</code> . Rien n'est auto-découvert.

Les trois mondes



- **Kernel** — ne connaît ni les modules ni les domaines. Change rarement.
- **Module** — ne connaît pas le projet. Se raccorde au kernel via des contrats uniquement.

- **Projet** — connaît tout, ne contient aucune logique métier. Câblage uniquement.

Comment lire ce guide. Développeurs / utilisateurs → chapitres 3, 4, 6, 7. Contributeurs internes → tout, surtout le chapitre 5. Les deux publics gagnent à lire le chapitre 2.

2. Pourquoi GDA — bénéfices et différences

Ce chapitre explique ce que GDA apporte que Laravel, Symfony ou Slim n'apportent pas, et les compromis.

2.1 Comparaison côte à côte

Aspect	Laravel / Symfony (typique)	PhpServicePlatform (GDA)
Coût de démarrage par requête	Tous les providers s'enregistrent à chaque requête	Seul le graphe de la route se charge
Coût d'une requête bloquée	L'app démarre souvent avant l'auth	La sécurité d'abord — refusé = 0 coût
Routes	Fichiers PHP / annotations	Déclarées dans <code>module.json</code>
Isolation des modules	Convention ; tout service est global	Imposée à l'exécution — lève si violée
Modélisation de l'argent	Flottants et modèles ORM fréquents	Objets valeur en centimes entiers
Couplage à l'infra	Façades / helpers globaux	Ports + DI uniquement ; pas de singleton global
Sûreté Swoole	Fuites d'état global entre requêtes	Noyau figé ; état requête jeté
Distribution native	Nécessite PHP + Composer + vendor/	Paquets OS natifs (.deb / .zip / .tar.gz)

2.2 Les bénéfices concrets

1 — Performance à l'usage

Une requête vers `/api/tasks` charge le module Task et ses `requires[]` — et rien d'autre. Sous OpenSwoole, le `CoreContainer` est construit une fois par worker ; seul le `ModuleContainer` à portée requête est reconstruit.

2 — Une sécurité non contournable

La SecurityGateway étant une étape avant ResolveStage/LoadStage, un 403/429 court-circuite la requête sans qu'aucun module soit construit. Impossible d'« oublier le middleware d'auth sur une route ».

3 — Architecture imposée par l'exécution, pas par la revue de code

Les cinq règles d'accès ne sont pas un guide de style — `bindInternal()` lève physiquement `ScopeViolationException` si un contrôleur atteint le repository d'un autre module.

4 — Testabilité par construction

Les services dépendent de ports et de contrats, jamais d'infra concrète : un test substitue `InMemoryRepository` / `FakeTransactionManager` sans aucun amorçage du framework.

5 — Déploiement honnête

Le kernel peut être livré en paquet OS natif sans `vendor/`, car le chargement d'env, le filet d'erreur et le runtime CLI sont du code maison à faibles dépendances.

Le compromis (soyons honnêtes) : GDA demande plus de structure en amont — un domaine par module, des contrats aux frontières, des manifestes, une config déclarée. Pour un petit site CRUD, c'est du surcoût. Le gain apparaît quand le système dépasse le point où un framework « mets ça dans un contrôleur » devient un enchevêtrement. Choisissez GDA pour des plateformes, pas pour des prototypes jetables.

3. Le cycle de vie du Kernel

Phase	Méthode	Ce qui se passe
1. Configurer	<code>Kernel::configure()</code> + <code>with*()</code>	Collecte ports, sécurité, modules, pipeline d'erreur, chemins. Déclaration pure.
2. Construire	<code>->build()</code>	Exécute le BootPipeline : valide la config et compile les manifestes . Compilation seule.
3. Matérialiser	premier <code>http()</code> / <code>cli()</code> / <code>workerLoop()</code>	Construit les pipelines, câble chaque module UNE fois, expose les services, puis fige le noyau.

Pourquoi séparer build et materialize ? `build()` est peu coûteux et exécutable en CI. `materialize()` est paresseux et spécifique à la surface : un processus HTTP ne lit jamais le manifeste des jobs. Le premier point d'entrée décide du `RuntimeMode`.

```
Kernel::configure()->withPorts([...])->withModules([...])->build();
$kernel->http();      // matérialise(RuntimeMode::Http), retourne HttpPipeline
$kernel->mode();      // RuntimeMode::Http (null avant tout point d'entrée)
```

4. Développeurs : utiliser le Kernel

Public : développeurs d'applications / utilisateurs.

4.1 Le constructeur fluide

app/bootstrap/base.php – base partagée (SANS ->build())

```
return Kernel::configure()
    ->withBasePath(dirname(__DIR__, 2))
    ->withPorts([
        DatabasePort::class => fn() => new PDODatabase(dsn: env('DB_DSN')),
        CachePort::class    => fn() => new InMemoryCache(),
    ])
    ->withSecurity([ new CsrfTokenLayer(exemptPaths: ['/api']) ])
    ->withModules([ CryptoProvider::class, DatabaseProvider::class ])
    ->withEssentialModules([ SessionProvider::class, CookieProvider::class ]);
```

projects/admin/bootstrap/app.php – projet concret

```
$builder = require __DIR__ . '/../../app/bootstrap/base.php';
return $builder
    ->withProjectPath(dirname(__DIR__))
    ->withModules([ AuthModule::class, InvoiceModule::class ])
    ->build();
```

Méthode	Fusion	Usage
<code>withPorts</code>	Fusionne ; dernière clé gagne	Infra. Une Closure = singleton paresseux (construit au 1er usage).
<code>withSecurity</code>	Ajoute	Couches, la moins coûteuse d'abord.
<code>withModules</code>	Ajoute + dédouble	Modules à la demande.
<code>withEssentialModules</code>	Ajoute + dédouble	Infra à portée requête nécessaire partout.

4.2 Ports vs essentiels vs à la demande

Règle : sans état + toujours → **port**. Avec état + toujours → **module essentiel**.
Nécessaire à certaines routes → **module à la demande** avec `requires[]`.

Ne jamais mettre d'état à portée requête sur un port (CoreContainer) — fuite entre requêtes sous OpenSwoole.

4.3 Points d'entrée

```
$kernel->http()           // → HttpPipeline::handle(Request): Response
$kernel->cli()             // → CliPipeline::run(argv): int
$kernel->workerLoop()      // → WorkerLoop::run(callable $puller, int $maxIterations = 0)
```

```
$kernel->container() // → CoreContainer (matérialise en surface Cli)
$kernel->requestTeardown() // après chaque requête sous OpenSwoole
```

4.4 TUTORIEL — créer un module de zéro (8 étapes)

Nous reconstruisons le plugin réel **Task**, dans `plugins/Task/` sous l'espace de noms `Plugins\Task\`, possédant le domaine `task.management`.

Étape 1 — Déclarer dans `module.json` (source unique de vérité)

```
{
  "name": "task", "solves": "task.management", "type": "module",
  "requires": ["database.query"],
  "exposes": ["Plugins\\Task\\API\\Contracts\\TaskServiceContract"],
  "routes": [
    { "method": "POST", "path": "/api/tasks",
      "handler": "Plugins\\Task\\Infrastructure\\Http\\TaskController@create" }
  ],
  "emits": ["task.created"], "config": []
}
```

Les routes sont déclarées ICI UNIQUEMENT. Chaque variable d'env lue doit figurer dans `config[]` sinon le démarrage échoue.

Étape 2 — L'entité de domaine (zéro import externe)

```
final class Task
{
    private array $domainEvents = [];
    private function __construct(
        private readonly TaskId $id, private string $title, private TaskStatus $status) {}

    public static function create(string $title): self { // ctor nommé – enregistre l'événement
        $t = new self(TaskId::generate(), $title, TaskStatus::PENDING);
        $t->domainEvents[] = new TaskCreatedDomainEvent($t->id, $title, new
\DateTimeImmutable());
        return $t;
    }
    public function markCompleted(): void {
        if ($this->status === TaskStatus::COMPLETED) throw new \DomainException('Déjà terminé');
        $this->status = TaskStatus::COMPLETED;
    }
    public function releaseEvents(): array {
        $e = $this->domainEvents; $this->domainEvents = []; return $e;
    }
}
```

Étape 3 — Le contrat publié (seul élément importable par d'autres modules)

```
interface TaskServiceContract {
    public function list(): array;
    public function create(CreateTaskDTO $dto): TaskDTO;
    public function find(string $id): ?TaskDTO;
    public function complete(string $id): ?TaskDTO;
```

```

    public function delete(string $id): bool;
}

```

Étape 4 — Le repository (DatabasePort uniquement, scope tenant)

```

final class TaskRepository {
    public function __construct(
        private readonly DatabasePort $db, private readonly Identity $identity) {}

    public function find(string $id): ?Task {
        try {
            $row = $this->db->queryOne('SELECT * FROM tasks WHERE id=:id AND tenant_id=:t',
                ['id'=>$id, 't'=>$this->identity->tenantId]);
        } catch (\PDOException $e) {
            throw new RepositoryException("find task [{$id}]", layer:'repository.task', previous:
            $e);
        }
        return $row === null ? null : TaskHydrator::hydrate($row);
    }
}

```

Étape 5 — Le service applicatif (forme transaction + événements obligatoire)

Ordre exact : début collection → début tx → travail → commit ; en échec rollback et discard ; diffusion des événements d'intégration UNIQUEMENT après commit, jamais dans `try{}` .

```

public function create(CreateTaskDTO $dto): TaskDTO {
    $this->collector->beginCollection();
    $this->transaction->begin();
    try {
        $task = Task::create($dto->title);
        foreach ($task->releaseEvents() as $event) $this->collector->collect($event);
        $this->repository->save($task);
        $this->transaction->commit();
    } catch (\Throwable $e) {
        $this->transaction->rollback();
        $this->collector->discard(); // ← pas d'événement fantôme au rollback
        throw new ServiceException('task.create.failed', layer:'service.task', previous:$e);
    }
    // APRÈS commit uniquement
    $created = $this->firstCreatedEvent($this->collector->release());
    if ($created !== null) {
        $this->eventBus->dispatch(new TaskCreatedIntegrationEvent(
            taskId: $created->taskId->value(), title: $created->title,
            occurredAt: $created->occurredAt->format(\DateTimeInterface::RFC3339)));
    }
    return TaskDTO::fromEntity($task);
}

```

Étape 6 — Le contrôleur (3 lignes max par méthode)

```

final class TaskController {
    public function __construct(private readonly TaskServiceContract $tasks) {} // contrat seul
    public function create(Request $request): Response {
        $task = $this->tasks->create(CreateTaskDTO::fromRequest($request));
        return Response::json(['data' => $task->toArray()], 201);
    }
}

```

```
}
}
```

Étape 7 — Le Provider : register() + boot()

```
final class Provider implements ModuleContract {
    public function solves(): string { return 'task.management'; }
    public function requires(): array { return [DatabasePort::class]; }
    public function exposes(): array { return [TaskServiceContract::class]; }

    public function register(ModuleContainer $c): void {
        $c->bindInternal(TaskRepository::class, static fn($c) =>
            new TaskRepository($c->make(DatabasePort::class), $c->make(Identity::class)));
        $c->bind(TaskServiceContract::class, static fn($c) =>
            new TaskService(
                repository: $c->make(TaskRepository::class),
                transaction: $c->make(TransactionManager::class),
                collector: $c->make(DomainEventCollector::class),
                eventBus: $c->make(EventBus::class),
                identity: $c->make(Identity::class)));
    }
    public function boot(HttpPipeline $http, CliPipeline $cli,
        WorkerPipeline $worker, EventBus $events): void { /* hooks + abonnements */ }
}
```

Étape 8 — Enregistrer le provider dans le projet

```
// projects/admin/bootstrap/app.php
return $builder->withModules([ \Plugins\Task\Provider::class ])->build();
```

C'est tout le module. `build()` le valide et le compile dans les manifestes. La première requête HTTP vers `/api/tasks` le charge à la demande.

4.5 Jobs Worker et commandes CLI

```
// module.json d'un job
{ "type": "job", "queue": "emails",
  "retry": { "max": 3, "strategy": "exponential", "jitter": true },
  "timeout": 30, "requires": ["mail.port"] }

final class SendInvoiceEmailJob implements JobContract {
    public function handle(JobPayload $payload): JobResult {
        return JobResult::success(['sent' => 1]); // lever pour relancer
    }
    public function failed(JobPayload $payload, \Throwable $e): void { /* après max relances */ }
}

// Commande CLI (php-io-cli, PAS Symfony)
final class SyncTasksCommand extends AbstractCommand {
    protected string $signature = 'tasks:sync';
    public function handle(): int { $this->output->success('Synchronisé.');
```

4.6 Tester un service (sans infra réelle)

```
$sut = new TaskService(new InMemoryTaskRepository(), new FakeTransactionManager(),
                      new DomainEventCollector(), new FakeIntegrationEventBus(),
                      Identity::asUser('user-1','tenant-abc'));
$result = $sut->create(new CreateTaskDTO('Écrire le guide'));
$this->assertTrue($txn->wasCommitted());
$bus->assertDispatched(TaskCreatedIntegrationEvent::class, times: 1);
```

5. Contributeurs : fonctionnement interne

Public : contributeurs internes. Mécanique interne de `src/Kernel/`.

5.1 build() vs materialize()

`build()` crée un `CoreContainer`, enregistre chaque port en `singleton` paresseux ou `instance` immédiate, puis exécute le `BootPipeline`. Kernel marqué `built` mais non figé ; aucun pipeline encore.

`materialize(RuntimeMode)` (privé, une fois) construit `EventBus`, `WorkerPipeline`, `HttpPipeline` (avec `SecurityGateway`), `CliPipeline`, `WorkerLoop` ; les réexpose dans le noyau ; parcourt chaque module, instancie le provider, appelle `boot()` ; puis `core->freeze()`.

Les trois pipelines sont construits car l'unique `boot()` peut hooker n'importe lequel — mais chaque pipeline diffère ses E/S disque jusqu'à sa première exécution.

5.2 Le pipeline de démarrage (9 étapes, échec rapide)

#	Étape	Vérifie / produit
1	ValidateConfig	Variables d'env présentes + typées
2	DetectConflicts	Pas deux modules pour le même <code>solves()</code>
3	DetectCycles	Pas de <code>requires[]</code> circulaire
4-7	Compile*Manifest	Manifestes service / route / job / commande
8	RegisterPorts	Vérifie les liaisons de ports
9	BindSecurity	Vérifie les couches de sécurité

5.3 Tracer une requête HTTP de bout en bout

Suivons `POST /api/tasks` :

1. **CorrelationIdStage** — génère/propage `X-Correlation-ID`.
2. **SecurityStage** — exécute la `SecurityGateway` (pare-feu → limite → CSRF → auth). En `deny`, retour immédiat — **aucun module chargé**. En `allow`, l'`Identity` est attachée. Hooks `after.security` ici.
3. **ResolveStage** — cherche la route dans `route-manifest.php` → domaine `task.management`, handler `TaskController@create`.
4. **LoadStage** — `DependencyGraphCalculator` calcule le graphe transitif (Task → Database). `OnDemandLoader` bâtit un `ModuleContainer` neuf, enregistre les services à portée requête (`Identity`, `DomainEventCollector`, `TransactionManager`), puis exécute chaque `register()` sous sa propre portée. Hooks `after.load`.
5. **ExecuteStage** — résout `TaskController` via `makeInScope(..., 'task.management')`, construit le DTO, appelle la méthode, obtient une `Response`. Hooks `after.execute`.

6. **ErrorStage** — englobe tout ; tout `Throwable` est classé et routé vers l' `ErrorPipeline` .
7. Le `ModuleContainer` est jeté — **zéro fuite d'état**. Sous Swoole, `requestTeardown()` est appelé.

Bandes de priorité : 1-9 système, 10-19 sécurité, 40-59 fonctionnalité, 80-99 observabilité.

5.4 Conteneurs et portées

- **CoreContainer** — durée de vie app, un par worker. `freeze()` le verrouille ; toute écriture lève `LogicException` . `getInstance()/setInstance()` désactivés.
- **ModuleContainer** — portée requête, neuf par requête/job, jeté après. `bindInternal()` impose la portée : résoudre une liaison interne de l'extérieur lève `ScopeViolationException` . `makeInScope($class, $domain)` résout avec portée explicite. `reset()` en fin de requête sous Swoole.

5.5 GUIDE PRATIQUE — étendre le kernel

Ajouter un port (ex. SearchPort)

1. Définir l'interface dans `src/Kernel/Ports/SearchPort.php` — méthodes seulement.
2. Fournir un adaptateur dans un plugin l'implémentant.
3. Le lier dans `base.php` : `->withPorts([SearchPort::class => fn() => new ElasticAdapter(env('ES_URL'))])` .
4. Les modules consommateurs injectent `SearchPort` par DI. Le kernel ne référence jamais l'adaptateur.

Un port est pour de l'infra SANS état à durée de vie app. Tout ce qui est à portée requête doit être un module essentiel ou à la demande.

Ajouter une étape de validation au démarrage

1. Créer une étape sous `src/Kernel/Boot/Stages/` ; lever `BootException` en échec.
2. L'enregistrer dans le `BootPipeline` dans le bon ordre (validation avant compilation).
3. La garder indépendante du domaine — manifestes/ports/config seulement.

5.6 Intérieur de la passerelle de sécurité

```
Requête → FirewallLayer → RateLimiterLayer → CsrfTokenLayer → [couche Auth] → pipeline
           ↓ deny(403)      ↓ deny(429)      ↓ deny(403)      ↓ deny(401)
           COÛT MODULE NUL à chaque refus
```

Une couche implémente `check(Request): SecurityVerdict` et **ne lève jamais**. L'autorisation (rôles/permissions) appartient à la couche *Service*. Le kernel ne fournit aucun validateur JWT — l'auth par jeton vient du module Auth du projet, sous forme de hook enregistré dans `boot()` .

6. Les cinq règles d'accès et la hiérarchie d'exceptions

Absolues, imposées à l'exécution par `ModuleContainer::bindInternal()`. Violation → `ScopeViolationException`.

```
Contrôleur → Service      (interface de contrat publiée UNIQUEMENT)
Service    → Repository ET Gateway (la SEULE couche appelant les deux)
Repository → DatabasePort UNIQUEMENT (pas de HTTP, pas de SDK)
Gateway    → SDK fournisseur UNIQUEMENT (pas de BD, pas de services)
Domaine    → RIEN D'EXTERNE          (zéro import hors de Domain/)
```

Couche	Lève	Traduit depuis	HTTP
Domaine	<code>\DomainException</code>	—	422
Service	<code>ServiceException</code>	tout Throwable capté	422/500
Repository	<code>RepositoryException</code>	<code>\PDOException</code>	500
Gateway	<code>GatewayException</code>	TOUTES les exceptions fournisseur	502
Sécurité	ne lève jamais — <code>deny()</code>	—	401/403

7. Référence rapide et listes de contrôle

Liste pour développeurs (livrer une fonctionnalité)

- Les routes vont dans `module.json`, jamais en PHP.
- Chaque variable d'env lue est déclarée dans `config[]`.
- Lire la config avec `env()`, jamais `getenv()`.
- L'argent est un objet valeur `Money` en centimes entiers — jamais flottant.
- Les événements d'intégration ne se diffusent qu'APRÈS le commit.
- Toujours `collector->discard()` dans le bloc catch.
- Contrôleurs : 3 lignes max — DTO → service → Response.
- Modules locaux dans `plugins/` avec l'espace de noms `Plugins\`.

Liste pour contributeurs (modifier le kernel)

- Garder le kernel indépendant du domaine.
- Nouvelle validation → étape à échec rapide dans le BootPipeline.
- Respecter `freeze()` — aucune écriture du noyau après materialize.
- L'état à portée requête ne vit jamais sur le CoreContainer.
- Pas de propriétés statiques dans les classes à portée requête.
- Traduire les exceptions fournisseur à leur couche — aucune ne s'échappe.

© AlfacodeTeam — PhpServicePlatform Kernel Guide · Expanded edition v2.0 · For internal contributors and framework users · EN/FR.